Lab 06: Introduction to OMNeT++ and INET

Duration: 1 hour 30 minutes | In-Lab Assessment

Tools: OMNeT++ and INET Framework

Objectives:

- Understand what OMNeT++ and INET are and why they are used.
- Learn basic OMNeT++ components: NED files, modules, gates and connections.
- Run a prebuilt INET example in the OMNeT++ IDE (Qtenv) and inspect results.
- Create and run a very small wired Ethernet network (2 hosts + switch) using INET, and verify connectivity with `PingApp`.

Short theory:

What is OMNeT++?
OMNeT++ is a discrete event simulation environment used to model communication networks, queuing networks and distributed systems. It provides an IDE (graphical) where you can write simple network descriptions (NED), implement behaviour in C++ if needed, run the simulation in a visual environment (Qtenv) and collect results.

What is INET?
INET is a large open source model library/framework built on OMNeT++. It provides pre-built, realistic protocol stacks (Ethernet, IP, TCP/UDP, wireless PHY/MAC, routing, lots of applications). Using INET saves you from implementing low level protocols and lets you focus on experiments. (See INET documentation and examples for details.)

Key OMNeT++ concepts:

- **NED files:** describe the network topology (modules and how they connect).
- **Modules:** building blocks. *Simple module* contains behaviour (in C++). *Compound module* contains submodules and connections.
- **Gates and connections:** modules talk through gates, connected by channels (wired links).
- **omnetpp.ini:** main configuration file (which network to run, parameter values, runtime choices).
- **Qtenv:** the graphical runtime where you can see packet animations. INET examples are prebuilt to use this. Wired link channels typically use a DatarateChannel with datarate/delay parameters.

Before the lab

- OMNeT++ installed (use official installer for your OS).
- INET framework downloaded and built inside OMNeT++ (import INET into the IDE and build; many INET examples appear in the Project Explorer).

- Familiarize: File → New → OMNeT++ Project, and how to open `omnetpp.ini` and NED files.
- If you have trouble installing, follow the documentation provided in the classroom

Part A: Run a ready-made INET example

1. Open OMNeT++ IDE.
2. In the Project Explorer open the INET examples folder (e.g. `inet/examples/ethernet/lans`). You should see NED networks and `omnetpp.ini` files. The repository's examples are a good starting point.
3. Right-click on an example's `omnetpp.ini` (or network NED) and choose `Run As` → `OMNeT++ Simulation` (or use Run Configurations).
4. In Qtenv you will see nodes and links. Press the green play button to let the simulation run (packets animate) or use step controls to single-step events.
5. Open the console/Status window — INET apps such as `PingApp` print ping results (RTT, packet loss). The simulation writes scalar/vector output files into the result directory specified in `omnetpp.ini`.

Part B: .ned file Explanation

File: `Networks.ned`

```
//
// Copyright (C) 2003 CTIE, Monash University
//
// SPDX-License-Identifier: LGPL-3.0-or-later
//
//

package inet.examples.ethernet.lans;

import inet.node.ethernet.EthernetHub;
import inet.node.ethernet.Eth100M;
import inet.node.ethernet.Eth10M;
import inet.node.ethernet.EthernetHost;
import inet.node.ethernet.EthernetSwitch;
import inet.physicallayer.wired.common.WireJunction;
import ned.DatarateChannel;

//
// Sample Ethernet LAN containing eight hosts, a switch and a bus.
//
network MixedLAN
{
    types:
        channel C extends Eth10M
        {
            length = 1cm;
        }
```

```
    submodules:
        tap1: WireJunction {
            parameters:
                @display("p=126,307");
        }
        tap2: WireJunction {
            parameters:
                @display("p=258,307");
        }
        tap3: WireJunction {
            parameters:
                @display("p=344,307");
        }
        tap4: WireJunction {
            parameters:
                @display("p=514,307");
        }
        tap5: WireJunction {
            parameters:
                @display("p=650,307");
        }
        tap6: WireJunction {
            parameters:
                @display("p=753,307");
        }
        busHostA: EthernetHost {
            parameters:
                eth.duplexMode = false;
                @display("p=126,200");
        }
        busHostB: EthernetHost {
            parameters:
                eth.duplexMode = false;
                @display("p=258,200");
        }
        busHostC: EthernetHost {
            parameters:
                eth.duplexMode = false;
                @display("p=514,400");
        }
        busHostD: EthernetHost {
            parameters:
                eth.duplexMode = false;
                @display("p=753,400");
        }
        switchHostA: EthernetHost {
            parameters:
                csmacdSupport = false;
                eth.duplexMode = true;
                @display("p=100,500");
        }
        switchHostB: EthernetHost {
            parameters:
```

```
                csmacdSupport = false;
                eth.duplexMode = true;
                @display("p=250,500");
        }
        switchHostC: EthernetHost {
            parameters:
                csmacdSupport = false;
                eth.duplexMode = true;
                @display("p=400,500");
        }
        switchHostD: EthernetHost {
            parameters:
                csmacdSupport = false;
                eth.duplexMode = true;
                @display("p=550,500");
        }
        switch: EthernetSwitch {
            parameters:
                eth[4].duplexMode = false;
                @display("p=344,400");
            gates:
                ethg[5];
        }
        hubHostA: EthernetHost {
            parameters:
                eth.duplexMode = false;
                @display("p=500,100");
        }
        hubHostB: EthernetHost {
            parameters:
                eth.duplexMode = false;
                @display("p=650,100");
        }
        hubHostC: EthernetHost {
            parameters:
                eth.duplexMode = false;
                @display("p=800,100");
        }
        hub: EthernetHub {
            parameters:
                @display("p=650,200");
        }
    connections:
        tap1.port++ <--> Eth10M {  length = 10m; } <--> tap2.port++;
        tap2.port++ <--> Eth10M {  length = 10m; } <--> tap3.port++;
        tap3.port++ <--> Eth10M {  length = 4m; } <--> tap4.port++;
        tap4.port++ <--> Eth10M {  length = 11m; } <--> tap5.port++;
        tap5.port++ <--> Eth10M {  length = 8m; } <--> tap6.port++;

        //half-duplex:
        tap1.port++ <--> C <--> busHostA.ethg;
        tap2.port++ <--> C <--> busHostB.ethg;
        tap3.port++ <--> C <--> switch.ethg[4];
```

```
        tap4.port++ <--> C <--> busHostC.ethg;
        tap5.port++ <--> C <--> hub.ethg++;
        tap6.port++ <--> C <--> busHostD.ethg;

        //full-duplex:
        switch.ethg[0] <--> Eth100M {  length = 3m; } <--> switchHostA.ethg;
        switch.ethg[1] <--> Eth100M {  length = 2m; } <--> switchHostB.ethg;
        switch.ethg[2] <--> Eth10M {  length = 4m; } <--> switchHostC.ethg;
        switch.ethg[3] <--> Eth100M {  length = 5m; } <--> switchHostD.ethg;

        //half-duplex:
        hub.ethg++ <--> Eth10M {  length = 3m; } <--> hubHostA.ethg;
        hub.ethg++ <--> Eth10M {  length = 4m; } <--> hubHostB.ethg;
        hub.ethg++ <--> Eth10M {  length = 5m; } <--> hubHostC.ethg;
}


//
// Sample Ethernet LAN: two hosts directly connected to each other
// via twisted pair.
//
network TwoHosts
{
    submodules:
        hostA: EthernetHost {
            parameters:
                @display("p=100,100");
        }
        hostB: EthernetHost {
            parameters:
                @display("p=300,100");
        }
    connections:
        hostA.ethg <--> {  delay = 0.5us; datarate = 100Mbps; } <--> hostB.ethg;
}


//
// Sample Ethernet LAN: four hosts connected to a switch.
//
network SwitchedLAN
{
    types:
        channel C extends DatarateChannel
        {
            delay = 0.1us;
            datarate = 100Mbps;
        }
```

```
    submodules:
        hostA: EthernetHost {
            parameters:
                csmacdSupport = false;
                eth.duplexMode = true;
                @display("p=250,100");
        }
        hostB: EthernetHost {
            parameters:
                csmacdSupport = false;
                eth.duplexMode = true;
                @display("p=400,200");
        }
        hostC: EthernetHost {
            parameters:
                csmacdSupport = false;
                eth.duplexMode = true;
                @display("p=250,300");
        }
        hostD: EthernetHost {
            parameters:
                csmacdSupport = false;
                eth.duplexMode = true;
                @display("p=100,200");
        }
        switch: EthernetSwitch {
            parameters:
                @display("p=250,200");
            gates:
                ethg[4];
        }
    connections:
        switch.ethg[0] <--> C <--> hostA.ethg;
        switch.ethg[1] <--> C <--> hostB.ethg;
        switch.ethg[2] <--> C <--> hostC.ethg;
        switch.ethg[3] <--> C <--> hostD.ethg;
}


//
// Sample Ethernet LAN: four hosts connected by a hub.
//
network HubLAN
{
    submodules:
        hostA: EthernetHost {
            parameters:
                eth.duplexMode = false;
                @display("p=250,100");
        }
        hostB: EthernetHost {
            parameters:
                eth.duplexMode = false;
```

```
                @display("p=400,200");
        }
        hostC: EthernetHost {
            parameters:
                eth.duplexMode = false;
                @display("p=250,300");
        }
        hostD: EthernetHost {
            parameters:
                eth.duplexMode = false;
                @display("p=100,200");
        }
        hub: EthernetHub {
            parameters:
                @display("p=250,200");
            gates:
                ethg[4];
        }
    connections:
        hub.ethg[0] <--> {  delay = 0.1us; datarate = 100Mbps; } <-->
           hostA.ethg;
        hub.ethg[1] <--> {  delay = 0.3us; datarate = 100Mbps; } <-->
           hostB.ethg;
        hub.ethg[2] <--> {  delay = 0.4us; datarate = 100Mbps; } <-->
           hostC.ethg;
        hub.ethg[3] <--> {  delay = 0.2us; datarate = 100Mbps; } <-->
           hostD.ethg;
}


//
// Sample Ethernet LAN: four hosts on a bus.
//
network BusLAN
{
    types:
        channel C extends Eth10M
        {
            length = 1cm;
        }
    submodules:
        hostA: EthernetHost {
            parameters:
                eth.duplexMode = false;
                @display("p=100,111");
        }
        hostB: EthernetHost {
            parameters:
                eth.duplexMode = false;
                @display("p=250,111");
        }
        hostC: EthernetHost {
            parameters:
```

```
                eth.duplexMode = false;
                @display("p=400,111");
        }
        hostD: EthernetHost {
            parameters:
                eth.duplexMode = false;
                @display("p=550,111");
        }
        tap1: WireJunction {
            parameters:
                @display("p=100,180");
        }
        tap2: WireJunction {
            parameters:
                @display("p=250,180");
        }
        tap3: WireJunction {
            parameters:
                @display("p=400,180");
        }
        tap4: WireJunction {
            parameters:
                @display("p=550,180");
        }
    connections:
        tap1.port++ <--> Eth10M <--> tap2.port++;
        tap2.port++ <--> Eth10M <--> tap3.port++;
        tap3.port++ <--> Eth10M <--> tap4.port++;
        hostA.ethg <--> C <--> tap1.port++;
        hostB.ethg <--> C <--> tap2.port++;
        hostC.ethg <--> C <--> tap3.port++;
        hostD.ethg <--> C <--> tap4.port++;
}

network SwitchedDuplexLAN
{
    types:
        channel ethernetline extends DatarateChannel
        {
            parameters:
                delay = 0.1us;
                datarate = 10Mbps;
        }

    submodules:
        hostA: EthernetHost {
            parameters:
                csmacdSupport = false;
                eth.duplexMode = true;
                @display("p=250,100");
        }
        hostB: EthernetHost {
            parameters:
```

```
                csmacdSupport = false;
                eth.duplexMode = true;
                @display("p=400,200");
        }
        hostC: EthernetHost {
            parameters:
                csmacdSupport = false;
                eth.duplexMode = true;
                @display("p=250,300");
        }
        hostD: EthernetHost {
            parameters:
                csmacdSupport = false;
                eth.duplexMode = true;
                @display("p=100,200");
        }
        switch: EthernetSwitch {
            parameters:
                @display("p=250,200");
            gates:
                ethg[4];
        }
    connections:
        switch.ethg[0] <--> ethernetline <--> hostA.ethg;
        switch.ethg[1] <--> ethernetline <--> hostB.ethg;
        switch.ethg[2] <--> ethernetline <--> hostC.ethg;
        switch.ethg[3] <--> ethernetline <--> hostD.ethg;
}
```

Explanation of important parts in `Networks.ned`

package & import lines

- `package inet.examples.ethernet.lans;` groups the NED definitions under a package name. This helps the IDE locate networks and avoids name clashes.
- `import inet.node.ethernet.EthernetHost;` etc. bring INET module types into scope so we can use them as submodules. These imported names (like `EthernetHost`) are prebuilt modules from INET.

network `MixedLAN` and `types`

- `types: channel C extends Eth10M { length = 1cm; }` declares a custom channel type named `C` which inherits properties from the INET `Eth10M` channel. You can then reuse `C` in many connections.
- Channels encapsulate physical link properties such as `datarate`, `delay` or `length`. These affect packet timing and throughput in simulations.

submodules: hosts, switch, hub, junctions

- `EthernetHost` is a prebuilt compound module representing a full host (NIC, protocol

stack, apps). Parameters like `eth.duplexMode = false` set the NIC to half-duplex (typical for bus/hub) or true for full-duplex (switch ports).

- `csmacdSupport` controls whether CSMA/CD behavior (carrier sense / collision detection) is used. For switched host links you often set it to `false` because switches avoid collisions.
- `EthernetSwitch` represents a layer-2 switch. The `gates: ethg[5];` declaration creates an array of Ethernet ports named `ethg` with 5 ports (indices 0..4).
- `WireJunction` is used for bus-style wiring / taps (it behaves like wires on a shared medium).
- `@display("p=x,y")` gives the coordinates for the graphical layout in Qtenv (purely visual).

connections: syntax and examples

- `hostA.ethg <-> delay = 0.5us; datarate = 100Mbps; <-> hostB.ethg;` connects two gates with an inline channel that sets delay and datarate.
- When you see `tap1.port++ <-> Eth10M length = 10m; <-> tap2.port++;`, the `port++` syntax tells NED to use the next available gate of a vector (useful when the junction has many ports).
- The operator `<->` denotes a bidirectional connection (symmetric).
- Comments such as `//half-duplex:` indicate the developer's intent: those connections model a shared-medium bus (half-duplex) while others model full-duplex switch links.

Multiple networks in one file

- This single NED file defines several example networks (`MixedLAN`, `TwoHosts`, `SwitchedLAN`, etc.). When you run a simulation you pick which network to instantiate from `omnetpp.ini`

Part C: A Simple Network Explanation

File: `Networks.ned/SwitchedLAN`

```
    network SwitchedLAN
{
    types:
        channel C extends DatarateChannel
        {
            delay = 0.1us;
            datarate = 100Mbps;
        }
    submodules:
        hostA: EthernetHost {
            parameters:
                csmacdSupport = false;
                eth.duplexMode = true;
                @display("p=250,100");
        }
        hostB: EthernetHost {
            parameters:
```

```
                csmacdSupport = false;
                eth.duplexMode = true;
                @display("p=400,200");
        }
        hostC: EthernetHost {
            parameters:
                csmacdSupport = false;
                eth.duplexMode = true;
                @display("p=250,300");
        }
        hostD: EthernetHost {
            parameters:
                csmacdSupport = false;
                eth.duplexMode = true;
                @display("p=100,200");
        }
        switch: EthernetSwitch {
            parameters:
                @display("p=250,200");
            gates:
                ethg[4];
        }
    connections:
        switch.ethg[0] <--> C <--> hostA.ethg;
        switch.ethg[1] <--> C <--> hostB.ethg;
        switch.ethg[2] <--> C <--> hostC.ethg;
        switch.ethg[3] <--> C <--> hostD.ethg;
}
```

SwitchedLAN Network Definition

```
network SwitchedLAN
{
```

This line declares a network named `SwitchedLAN`. It represents a switched Local Area Network where multiple Ethernet hosts communicate through a central switch.

Custom Channel Definition

```
    types:
        channel C extends DatarateChannel
        {
            delay = 0.1us;
            datarate = 100Mbps;
        }
```

Channel Type C The custom channel `C` extends `DatarateChannel`, allowing explicit configuration of delay and bandwidth for Ethernet links.

Delay The propagation delay is set to `0.1 microseconds`, modeling a short-distance LAN cable.

Data Rate The data rate is configured as `100 Mbps`, which corresponds to Fast Ethernet.

Submodules Description

```
    submodules:
        hostA: EthernetHost {
            parameters:
                csmacdSupport = false;
                eth.duplexMode = true;
                @display("p=250,100");
        }
```

Ethernet Host A `hostA` is an instance of `EthernetHost`. It represents an end device connected to the LAN.

CSMA/CD Support `csmacdSupport = false` disables collision detection, which is unnecessary in a switched, full-duplex Ethernet network.

Duplex Mode `eth.duplexMode = true` enables full-duplex communication, allowing simultaneous transmission and reception.

Display Annotation The `@display` parameter specifies the visual position of the host in the OMNeT++ GUI.

Additional Ethernet Hosts

```
        hostB: EthernetHost {
            parameters:
                csmacdSupport = false;
                eth.duplexMode = true;
                @display("p=400,200");
        }
        hostC: EthernetHost {
            parameters:
                csmacdSupport = false;
                eth.duplexMode = true;
                @display("p=250,300");
        }
        hostD: EthernetHost {
            parameters:
                csmacdSupport = false;
                eth.duplexMode = true;
                @display("p=100,200");
        }
```

Each of these hosts (`hostB`, `hostC`, and `hostD`) is configured identically to `hostA`. All hosts operate in full-duplex mode and are visually positioned around the switch.

Ethernet Switch Configuration

```
        switch: EthernetSwitch {
            parameters:
                @display("p=250,200");
            gates:
                ethg[4];
        }
```

Switch Module The `EthernetSwitch` acts as the central forwarding device in the network.

Gate Array The switch defines four Ethernet gates (`ethg[4]`), each corresponding to a physical switch port used to connect one host.

Display Position The display annotation places the switch at the center of the topology in the simulation environment.

Connections Description

```
    connections:
        switch.ethg[0] <--> C <--> hostA.ethg;
        switch.ethg[1] <--> C <--> hostB.ethg;
        switch.ethg[2] <--> C <--> hostC.ethg;
        switch.ethg[3] <--> C <--> hostD.ethg;
}
```

Host-to-Switch Links Each host is connected to a unique switch port using the custom channel `C`. The bidirectional operator (`<->`) ensures symmetric communication.

Dedicated Links Since each host has its own point-to-point link to the switch, no medium is shared among hosts.

Collision-Free Communication Because the links are full-duplex and point-to-point, collisions cannot occur, resulting in higher throughput and predictable performance.

Part D: .ini File Explanation

File: `omnetpp.ini`

```
[General]
sim-time-limit = 10s
**.vector-recording = false
description = "(abstract)"

[Config SimpleLan1]
network = SwitchedLAN
**.hostA.cli.destAddress = ""
**.cli.destAddress = "hostA"
**.cli.sendInterval = exponential(1s)
**.cli.reqLength = intuniform(50,1400)*1B
**.cli.respLength = intWithUnit(truncnormal(3000B,3000B))
**.switch*.bridging.typename = "MacRelayUnit"
```

Line-by-line explanation

```
[General]
sim-time-limit = 10s
```

Stops the simulation at 10 simulated seconds.

```
**.vector-recording = false
```

Globally disables vector recording (time-series vectors). You can selectively override this for specific modules.

```
description = "(abstract)"
```

A human-readable description; no effect on simulation logic.

```
[Config SimpleLan1]
network = SwitchedLAN
```

Defines a configuration named `SimpleLan1` and selects the `SwitchedLAN` network (the NED provided earlier).

```
**.hostA.cli.destAddress = ""
```

Sets the `destAddress` parameter of the `cli` submodule inside hostA to an empty string (likely means "not set").

```
**.cli.destAddress = "hostA"
```

Sets `destAddress` for all modules named `cli` to `"hostA"`. Note specificity/precedence rules: more specific keys (e.g., `**.hostA.cli.*`) override less specific.

```
**.cli.sendInterval = exponential(1s)
```

Sets the send interval distribution for all `cli` modules to an exponential distribution with mean 1s.

```
**.cli.reqLength = intuniform(50,1400)*1B
```

Request length is a uniformly random integer between 50 and 1400 bytes.

```
**.cli.respLength = intWithUnit(truncnormal(3000B,3000B))
```

Response length drawn from a truncated normal distribution with mean=3000B and std-dev=3000B — this is a very wide spread; truncated to avoid negative sizes. `intWithUnit` converts to integer unit.

```
**.switch*.bridging.typename = "MacRelayUnit"
```

Sets the bridging implementation (typename) used by switch modules that match the wildcard. Implementation names and exact parameter keys depend on the INET version.

Rules & patterns to remember

- Wildcards: `**` matches any module path; `*` matches one level. Use them to target many modules quickly.
- Specificity: a parameter targeted at `**.hostA.cli.param` is more specific than `**.cli.param` and will override it.
- Units: always include units for sizes and times (e.g., `B`, `s`, `ms`, `bps`).
- Check module names: verify that modules (like `cli`) and parameters (`reqLength`) exist in your INET version.

Concrete modifications and extensions

Below are ready-to-use INI lines. Put them under the appropriate `[Config ...]` section.

1) Change sender and destination

```
# Make hostA send to hostB
**.hostA.cli.destAddress = "hostB"

# Make hostB send to hostC; hostC -> hostD
**.hostB.cli.destAddress = "hostC"
**.hostC.cli.destAddress = "hostD"

# Set all CLI modules to send to hostD
**.cli.destAddress = "hostD"
```

2) Change send rate / inter-send distribution

```
# Deterministic 0.2s interval for hostA
**.hostA.cli.sendInterval = 0.2s

# Poisson-like with mean 0.5s for hostB
**.hostB.cli.sendInterval = exponential(0.5s)

# Different distributions per host
**.hostA.cli.sendInterval = uniform(0.1s,0.5s)
**.hostB.cli.sendInterval = exponential(1s)
**.hostC.cli.sendInterval = normal(0.5s,0.1s)  # be careful with
   negatives
```

3) Change packet sizes

```
# Fixed request size 500 bytes
**.cli.reqLength = 500B

# Uniform request size between 100 and 1500 bytes
**.cli.reqLength = intuniform(100,1500)*1B

# Response size: truncated normal mean 2000B stddev 200B
**.cli.respLength = intWithUnit(truncnormal(2000B,200B))

# Deterministic response size
**.cli.respLength = 1200B
```

4) Start/stop times (stagger traffic)

```
# Start and stop times for hostA
**.hostA.cli.startTime = 0.5s
**.hostA.cli.stopTime  = 8s

# Staggered starts
**.hostB.cli.startTime = 1s
**.hostC.cli.startTime = 2s
```

5) Multiple apps per host (example pattern)

```
# If your host supports multiple apps, configure them per-host
**.hostA.numApps = 2
**.hostA.app[0].typename = "UdpBasicApp"
```

```
**.hostA.app[0].destAddresses = "hostB"
**.hostA.app[0].sendInterval = 0.1s

**.hostA.app[1].typename = "UdpBasicApp"
**.hostA.app[1].destAddresses = "hostC"
**.hostA.app[1].sendInterval = 0.5s
```

(Replace app, numApps, and parameter names with the ones in your INET version.)

6) Change switch behavior

```
# Use a different bridge implementation (check your INET)
**.switch*.bridging.typename = "LearningBridge"

# Example MAC aging time (name may differ by INET)
**.switch*.bridging.macEntryTimeout = 300s
```

7) Enable recording selectively

```
# Global off, but enable for hostA and the switch
**.vector-recording = false
**.hostA.*.vector-recording = true
**.switch*.vector-recording = true

# Or enable globally
**.vector-recording = true
```

8) Enable pcap / packet capture (INET-specific)

```
# Example conceptual enabling of pcap; exact parameter path depends on
    INET version
**.host*.ppp.interface[0].pcapRecorder.enable = true
```

(Inspect module tree in OMNeT++ GUI to find exact recorder module name.)

9) Repeatable runs / RNG seeds

```
# Repeatable experiments
seed-set = 42
repeat = 5
```

10) Per-link bandwidth (if channel parameters are addressable)

```
# Only valid if channel instances/parameters are accessible via INI
**.C.datarate = 1Gbps
**.C.delay = 1us
```

(If this does not work, define multiple channel types inside NED and use them there.)

11) Addressing and auto-configurator

```
# Use INET configurator features (parameter names depend on INET)
*.configurator.assignAddresses = true
*.configurator.addStaticRoutes = true
```

12) Emulate link failure (conceptual)

```
# You can schedule start/stop or write a control module that toggles
   gates.
# Alternatively, use apps' startTime/stopTime to emulate outages.
**.hostA.cli.stopTime = 5s  # hostA stops sending at 5s
```

Example scenarios

A. HostA → HostB, high-rate small packets

```
**.hostA.cli.destAddress = "hostB"
**.hostA.cli.sendInterval = 0.01s
**.hostA.cli.reqLength = intuniform(64,128)*1B
**.hostA.cli.respLength = 128B
```

B. Many sporadic large responses to hostA

```
**.cli.destAddress = "hostA"
**.cli.sendInterval = exponential(2s)
**.cli.reqLength = 100B
**.cli.respLength = intWithUnit(truncnormal(3000B,500B))
```

C. Turn on vector recording only for hostA and the switch

```
**.vector-recording = false
**.hostA.*.vector-recording = true
**.switch*.vector-recording = true
```

Troubleshooting tips & best practices

- If `destAddress` is empty, the app may not send. Always set explicit destinations or use broadcast where appropriate.
- Verify module and parameter names for your INET version — names vary across versions.
- Use the OMNeT++ inspector to view full module tree and available parameters.
- Keep units explicit (`B`, `s`, `ms`, `bps`).
- For reproducibility, set `seed-set` and `repeat`.
- When using distributions like `normal(...)` or `truncnormal(...)`, ensure stddev is reasonable to avoid degenerate behavior.
- If switching bridging implementations, confirm the targeted `typename` exists in your INET release.

Part E: GUI Usage

The core logic stays the same, but instead of coding you can drag and drop different components, modules, submodules etc. Will be shown during the lab.

Debugging tips

- **No packets visible/instant simulation end:** check `sim-time-limit` in `omnetpp.ini` and ensure startTime of apps is within limit.
- **IP not assigned / hosts cannot ping:** include an `Ipv4NetworkConfigurator` in NED or use `flatNetworkConfigurator` to set IPs automatically.
- **Wrong import or module name:** INET versions move module classes between packages. If an import fails, search for the module name in your INET `src` tree (IDE search).
- **Build errors:** ensure INET is built first and your project references it (Project → Properties → Project References).